

SERVICIOS DE RED EN UBUNTU SERVER

FTP | IIS | SSH | DNS | HTTP | DHCP

Administración y Configuración de Servicios en Sistemas Linux

Asignatura: Servicios en Red

ÍNDICE DE CONTENIDOS

1. Introducción a los Servicios de Red
2. Servicio FTP (File Transfer Protocol)
 - 2.1 ¿Qué es FTP?
 - 2.2 Historia y evolución
 - 2.3 Modos de conexión: activo y pasivo
 - 2.4 Instalación de vsftpd en Ubuntu Server
 - 2.5 Configuración de vsftpd
 - 2.6 Usuarios y permisos
 - 2.7 FTP con TLS/SSL (FTPS)
 - 2.8 Comandos FTP más importantes
 - 2.9 Alternativas: ProFTPD y Pure-FTPd
3. Servidor Web IIS en Ubuntu (Apache como equivalente)
 - 3.1 ¿Qué es IIS y sus equivalentes en Linux?
 - 3.2 Instalación de Apache2
 - 3.3 Configuración de VirtualHosts
 - 3.4 Módulos de Apache
 - 3.5 HTTPS con Let's Encrypt
 - 3.6 PHP y bases de datos (LAMP)
 - 3.7 Comparativa IIS vs Apache vs Nginx
4. Servicio SSH (Secure Shell)
 - 4.1 ¿Qué es SSH?
 - 4.2 Historia y versiones del protocolo
 - 4.3 Instalación y activación
 - 4.4 Configuración del servidor SSH
 - 4.5 Autenticación por clave pública/privada
 - 4.6 Hardening y seguridad SSH
 - 4.7 Túnel SSH y reenvío de puertos
 - 4.8 SCP y SFTP
5. Servicio DNS (Domain Name System)
 - 5.1 ¿Qué es DNS?
 - 5.2 Tipos de registros DNS
 - 5.3 Instalación de BIND9 en Ubuntu Server
 - 5.4 Configuración de una zona DNS
 - 5.5 DNS autoritativo y recursivo
 - 5.6 Pruebas y verificación
6. Servicio HTTP (HyperText Transfer Protocol)

- 6.1 ¿Qué es HTTP?
- 6.2 HTTP vs HTTPS
- 6.3 Cabeceras HTTP relevantes
- 6.4 Códigos de estado HTTP
- 6.5 Configuración de seguridad HTTP en Apache
- 7. Servicio DHCP (Dynamic Host Configuration Protocol)
 - 7.1 ¿Qué es DHCP?
 - 7.2 Instalación de isc-dhcp-server
 - 7.3 Configuración de rangos y opciones
 - 7.4 Reservas de IP (direcciones estáticas)
 - 7.5 Verificación y logs
- 8. Comparativa de seguridad entre servicios
- 9. Buenas prácticas y recomendaciones
- 10. Referencias y bibliografía

1. INTRODUCCIÓN A LOS SERVICIOS DE RED

Ubuntu Server es una de las distribuciones Linux más utilizadas en entornos empresariales y de producción para la gestión de servicios de red. Su estabilidad, soporte a largo plazo (LTS) y extensa documentación lo convierten en la plataforma ideal para desplegar servicios como FTP, servidores web y SSH.

Un servicio de red es un programa o conjunto de programas que se ejecutan en un servidor y ofrecen funcionalidades a otros equipos (clientes) a través de la red. Estos servicios siguen el modelo cliente-servidor y se comunican mediante protocolos estandarizados definidos en documentos RFC (Request for Comments) publicados por la IETF.

Los servicios de red fundamentales en la administración de sistemas son:

- FTP (File Transfer Protocol): transferencia de archivos entre sistemas.
- SSH (Secure Shell): acceso remoto seguro y transferencia de archivos cifrada.
- HTTP/HTTPS (HyperText Transfer Protocol): servicio web.
- DNS (Domain Name System): resolución de nombres de dominio.
- DHCP (Dynamic Host Configuration Protocol): asignación dinámica de IPs.

En este documento se estudiarán en profundidad FTP, IIS (y su equivalente en Linux, Apache), SSH, DNS, HTTP y DHCP, sus instalaciones, configuraciones, parámetros de seguridad y buenas prácticas.

2. SERVICIO FTP (FILE TRANSFER PROTOCOL)

2.1 ¿Qué es FTP?

FTP (File Transfer Protocol) es un protocolo de red estándar utilizado para la transferencia de archivos entre un cliente y un servidor a través de una red TCP/IP. Fue definido originalmente en el RFC 959 en 1985, aunque sus orígenes se remontan al RFC 114 de 1971. Es uno de los protocolos de aplicación más antiguos de Internet y, aunque ha sido ampliamente sustituido por alternativas seguras como SFTP y FTPS, sigue utilizándose en multitud de entornos.

FTP opera sobre la capa de aplicación del modelo OSI y utiliza el protocolo de transporte TCP. A diferencia de la mayoría de los protocolos, FTP utiliza dos conexiones TCP separadas:

- Canal de control (puerto 21): se usa para enviar comandos y recibir respuestas entre cliente y servidor. Esta conexión permanece activa durante toda la sesión.
- Canal de datos (puerto 20 en modo activo, o un puerto dinámico en modo pasivo): se usa para la transferencia real de archivos y listados de directorios. Se abre y cierra para cada transferencia.

2.2 Historia y evolución del protocolo FTP

FTP ha evolucionado considerablemente desde sus orígenes. La siguiente tabla resume su historia:

Año	RFC	Descripción
1971	RFC 114	Primera especificación de FTP
1973	RFC 454	Segunda versión mejorada
1980	RFC 765	Versión revisada con TCP/IP
1985	RFC 959	Especificación actual y más usada de FTP
1997	RFC 2228	Extensiones de seguridad para FTP
1998	RFC 2389	Mecanismo de negociación de características
1999	RFC 2428	Soporte para IPv6 y modo pasivo extendido
2007	RFC 4217	Asegurando FTP con TLS

2.3 Modos de conexión: Activo y Pasivo

Modo Activo (PORT)

En el modo activo, el cliente abre un puerto aleatorio mayor que 1023 y notifica al servidor mediante el comando PORT. El servidor entonces inicia la conexión de datos desde su puerto 20 hacia el puerto indicado por el cliente.

Ventaja: simple de configurar en el servidor. Desventaja: los cortafuegos del cliente frecuentemente bloquean las conexiones entrantes desde el servidor, lo que provoca fallos en la transferencia.

Modo Pasivo (PASV)

En el modo pasivo, es el cliente quien inicia la conexión de datos. El servidor abre un puerto aleatorio y notifica al cliente con el comando PASV. El cliente entonces se conecta a ese puerto para la transferencia.

Ventaja: es compatible con la mayoría de cortafuegos y NAT, ya que el cliente es quien inicia ambas conexiones. Este modo es el preferido y más usado en la actualidad.

Característica	Modo Activo	Modo Pasivo
Quién inicia datos	Servidor (desde puerto 20)	Cliente
Puerto servidor datos	20	Dinámico (1024–65535)
Compatibilidad NAT	Problemática	Excelente
Compatibilidad cortafuegos	Baja	Alta
Uso recomendado	Redes internas	Internet / producción

2.4 Instalación de vsftpd en Ubuntu Server

vsftpd (Very Secure FTP Daemon) es el servidor FTP más utilizado en distribuciones Linux por su seguridad, rendimiento y facilidad de configuración. Para instalarlo en Ubuntu Server:

```
sudo apt update
sudo apt install vsftpd -y
```

Una vez instalado, verificamos que el servicio está activo:

```
sudo systemctl status vsftpd
sudo systemctl enable vsftpd      # habilitar al inicio del sistema
sudo systemctl start vsftpd       # iniciar el servicio
```

Podemos comprobar que vsftpd está escuchando en el puerto 21:

```
sudo ss -tlnp | grep 21
sudo netstat -tlnp | grep vsftpd
```

2.5 Configuración de vsftpd

El archivo principal de configuración se encuentra en `/etc/vsftpd.conf`. A continuación se detallan los parámetros más importantes:

```
# /etc/vsftpd.conf - Configuración principal
listen=YES                # vsftpd escucha conexiones IPv4
listen_ipv6=NO            # deshabilitar IPv6 si no se usa
anonymous_enable=NO      # IMPORTANTE: deshabilitar acceso anónimo
local_enable=YES          # permitir usuarios locales del sistema
write_enable=YES          # permitir escritura (subida de archivos)
local_umask=022           # permisos por defecto de archivos subidos
chroot_local_user=YES     # encerrar usuarios en su directorio home
pasv_enable=YES           # habilitar modo pasivo
pasv_min_port=10000
pasv_max_port=10100
```

Después de modificar el archivo de configuración, siempre hay que reiniciar el servicio:

```
sudo systemctl restart vsftpd
```

2.6 Usuarios y permisos en FTP

La gestión de usuarios es uno de los aspectos más importantes en la administración de un servidor FTP. vsftpd puede usar los usuarios del sistema operativo o una lista de usuarios virtuales.

Creación de un usuario FTP dedicado

```
sudo adduser ftpuser --shell /usr/sbin/nologin
sudo mkdir -p /home/ftpuser/ftp/upload
sudo chown nobody:nogroup /home/ftpuser/ftp
sudo chmod a-w /home/ftpuser/ftp
sudo chown ftpuser:ftpuser /home/ftpuser/ftp/upload
```

Lista de usuarios permitidos

Para restringir el acceso FTP solo a ciertos usuarios, se puede usar el parámetro `userlist_enable`:

```
userlist_enable=YES
userlist_file=/etc/vsftpd.userlist
userlist_deny=NO      # la lista es de usuarios PERMITIDOS
echo 'ftpuser' | sudo tee -a /etc/vsftpd.userlist
```

2.7 FTP seguro con TLS/SSL (FTPS)

FTP en su forma básica transmite datos en texto plano, incluyendo credenciales. Para solucionar esto, se utiliza FTPS, que cifra la comunicación mediante TLS/SSL. Esto está definido en el RFC 4217.

Generación de certificado autofirmado

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/ssl/private/vsftpd.key \
    -out /etc/ssl/certs/vsftpd.crt
```

Configuración de TLS en vsftpd

```
ssl_enable=YES
ssl_tlsv1=YES
ssl_sslv2=NO          # deshabilitar SSLv2 (inseguro)
ssl_sslv3=NO          # deshabilitar SSLv3 (inseguro)
rsa_cert_file=/etc/ssl/certs/vsftpd.crt
rsa_private_key_file=/etc/ssl/private/vsftpd.key
force_local_data_ssl=YES    # forzar cifrado en datos
force_local_logins_ssl=YES  # forzar cifrado en login
```

2.8 Comandos FTP más importantes

Comando	Descripción
open host	Conectar a un servidor FTP
user nombre	Autenticarse con un usuario
ls / dir	Listar archivos del directorio remoto
cd directorio	Cambiar directorio remoto

lcd directorio	Cambiar directorio local
get archivo	Descargar un archivo del servidor
put archivo	Subir un archivo al servidor
mget *.ext	Descargar múltiples archivos
mput *.ext	Subir múltiples archivos
delete archivo	Eliminar un archivo remoto
mkdir directorio	Crear un directorio remoto
rmdir directorio	Eliminar un directorio remoto
binary / ascii	Cambiar modo de transferencia
passive	Alternar modo pasivo/activo
bye / quit	Cerrar la conexión FTP

2.9 Alternativas: ProFTPD y Pure-FTPd

ProFTPD

ProFTPD es otro popular servidor FTP para Linux. Su archivo de configuración es similar en sintaxis a Apache, lo que facilita su administración a quienes ya conocen dicho servidor web. Soporta usuarios virtuales mediante SQL, autenticación LDAP y módulos personalizados.

```
sudo apt install proftpd -y
# Archivo de configuración: /etc/proftpd/proftpd.conf
```

Pure-FTPd

Pure-FTPd es conocido por su simplicidad y alta seguridad. No tiene archivo de configuración tradicional; en cambio, se configura mediante opciones en la línea de comandos o archivos separados. Soporta usuarios virtuales, cuotas de disco y cifrado TLS.

```
sudo apt install pure-ftpd -y
```


3. SERVIDOR WEB: IIS Y SU EQUIVALENTE EN UBUNTU (APACHE2)

3.1 ¿Qué es IIS y los servidores web en Linux?

IIS (Internet Information Services) es el servidor web y de aplicaciones desarrollado por Microsoft para sus sistemas operativos Windows Server. Ofrece soporte para HTTP, HTTPS, FTP, SMTP y otros protocolos. Sin embargo, IIS no está disponible de forma nativa para Linux ni Ubuntu Server.

En entornos Ubuntu Server, los equivalentes funcionales más utilizados son:

- Apache HTTP Server (Apache2): el servidor web más utilizado del mundo. Modular, estable y con soporte para múltiples lenguajes de programación.
- Nginx: servidor web y proxy inverso de alto rendimiento, ideal para sitios con alto tráfico.
- Lighttpd: servidor web ligero orientado a entornos con recursos limitados.

3.2 Instalación de Apache2 en Ubuntu Server

```
sudo apt update
sudo apt install apache2 -y
```

Verificamos el estado e iniciamos el servicio:

```
sudo systemctl status apache2
sudo systemctl enable apache2
sudo systemctl start apache2
```

Comprobamos que Apache escucha en el puerto 80:

```
sudo ss -tlnp | grep 80
```

Podemos verificar desde el navegador accediendo a `http://IP_DEL_SERVIDOR`. Si vemos la página de bienvenida de Apache2, el servidor está funcionando correctamente.

El cortafuegos UFW debe permitir el tráfico web:

```
sudo ufw allow 'Apache Full' # permite HTTP (80) y HTTPS (443)
sudo ufw status
```

3.3 Estructura de directorios de Apache2

Ruta	Descripción
/etc/apache2/	Directorio principal de configuración
/etc/apache2/apache2.conf	Archivo de configuración global
/etc/apache2/sites-available/	Definiciones de VirtualHosts disponibles
/etc/apache2/sites-enabled/	VirtualHosts activos (enlaces simbólicos)
/etc/apache2/mods-available/	Módulos disponibles
/etc/apache2/mods-enabled/	Módulos activos
/var/www/html/	Directorio raíz por defecto del servidor web
/var/log/apache2/	Registros de acceso y errores

3.4 Configuración de VirtualHosts

Los VirtualHosts permiten alojar múltiples sitios web en un mismo servidor físico. Apache2 incluye una configuración por defecto en `/etc/apache2/sites-available/000-default.conf`.

```
sudo nano /etc/apache2/sites-available/miweb.conf
<VirtualHost *:80>
    ServerName www.miweb.com
    ServerAlias miweb.com
    DocumentRoot /var/www/miweb
    ErrorLog ${APACHE_LOG_DIR}/miweb_error.log
    CustomLog ${APACHE_LOG_DIR}/miweb_access.log combined
</VirtualHost>
```

Creamos el directorio y habilitamos el sitio:

```
sudo mkdir -p /var/www/miweb
sudo chown -R www-data:www-data /var/www/miweb
echo '<h1>Bienvenido a miweb.com</h1>' | sudo tee /var/www/miweb/index.html
sudo a2ensite miweb.conf
sudo systemctl reload apache2
```

3.5 Módulos de Apache2

Módulo	Función
mod_rewrite	Reescritura de URLs (SEO, redirecciones)
mod_ssl	Soporte HTTPS/TLS
mod_headers	Manipulación de cabeceras HTTP
mod_deflate	Compresión gzip de contenido
mod_expires	Control de caché del navegador
mod_auth_basic	Autenticación HTTP básica
mod_proxy	Funciones de proxy inverso
mod_php	Soporte para PHP
mod_status	Página de estado del servidor
mod_security	Cortafuegos de aplicaciones web (WAF)

```
sudo a2enmod rewrite ssl headers deflate
sudo systemctl restart apache2
```

3.6 HTTPS con Let's Encrypt (Certbot)

HTTPS es imprescindible en cualquier sitio web en producción. Let's Encrypt ofrece certificados SSL/TLS gratuitos y automatizados. En Ubuntu Server se usa la herramienta Certbot.

```
sudo apt install certbot python3-certbot-apache -y
sudo certbot --apache -d www.miweb.com -d miweb.com
```

Certbot modifica automáticamente la configuración de Apache y renueva los certificados antes de que expiren. Para verificar la renovación automática:

```
sudo certbot renew --dry-run
```

3.7 LAMP Stack: Apache + MySQL + PHP

Una de las configuraciones más habituales en servidores Ubuntu es la pila LAMP (Linux, Apache, MySQL, PHP), que proporciona un entorno completo para aplicaciones web dinámicas.

```
sudo apt install php libapache2-mod-php php-mysql -y
sudo apt install mysql-server -y
sudo mysql_secure_installation # configuración segura inicial
```

Para comprobar que PHP funciona con Apache, crear un archivo de prueba:

```
echo '<?php phpinfo(); ?>' | sudo tee /var/www/html/info.php
```

Acceder a http://IP_DEL_SERVIDOR/info.php mostrará información completa sobre PHP.

3.8 Comparativa IIS vs Apache vs Nginx

Característica	IIS	Apache2	Nginx
Sistema operativo	Windows	Linux/Windows/Mac	Linux/Windows/Mac
Coste	Incluido en Windows	Gratuito (código abierto)	Gratuito (código abierto)
Rendimiento alto tráfico	Bueno	Bueno	Excelente
Consumo de memoria	Medio	Medio	Bajo
Configuración	GUI + XML	Archivos .conf	Archivos .conf
VirtualHosts	Sí (Sites)	Sí	Sí (Server Blocks)
Soporte PHP	Vía FastCGI	mod_php / FastCGI	FastCGI
Módulos	Sí	Sí (extenso)	Limitado
Comunidad y soporte	Microsoft oficial	Muy amplia	Amplia

4. SERVICIO SSH (SECURE SHELL)

4.1 ¿Qué es SSH?

SSH (Secure Shell) es un protocolo de red criptográfico que permite la comunicación segura entre dos sistemas a través de una red insegura. Fue diseñado como reemplazo seguro de protocolos inseguros como Telnet, rsh y rlogin, que transmitían datos en texto plano.

SSH proporciona tres garantías fundamentales de seguridad:

- **Confidencialidad:** toda la comunicación está cifrada, por lo que un atacante que intercepte el tráfico no podrá leer su contenido.
- **Integridad:** los datos no pueden ser modificados en tránsito sin que sea detectado.
- **Autenticación:** tanto el cliente como el servidor verifican la identidad del otro antes de establecer la sesión.

SSH se utiliza principalmente para:

- Acceso remoto seguro a la línea de comandos de un servidor.
- Transferencia segura de archivos (SCP y SFTP).
- Tunelización y reenvío de puertos (port forwarding).
- Ejecución remota de comandos y scripts.
- Montaje de sistemas de archivos remotos (SSHFS).

4.2 Historia y versiones del protocolo SSH

SSH fue creado en 1995 por Tatu Ylönen, investigador de la Universidad Tecnológica de Helsinki, tras sufrir un ataque de captura de contraseñas en su red universitaria.

Versión	Descripción
SSH-1 (1995)	Primera versión. Usa RSA para intercambio de claves. Tiene vulnerabilidades conocidas; NO debe usarse.
SSH-2 (1996–2006)	Versión actual. Definida en RFC 4251–4256. Usa DH para intercambio de claves, soporta DSA, RSA y ECDSA. Incompatible con SSH-1.
OpenSSH	Implementación de código abierto mantenida por el proyecto OpenBSD. Es la implementación más utilizada en Linux y macOS.

4.3 Instalación y activación del servidor SSH

En Ubuntu Server, el servidor SSH (OpenSSH) puede estar preinstalado, pero si no lo está:

```
sudo apt update
sudo apt install openssh-server -y
```

Gestión del servicio:

```
sudo systemctl status ssh
sudo systemctl enable ssh    # iniciar automáticamente en el arranque
sudo systemctl start ssh
sudo systemctl restart ssh   # aplicar cambios de configuración
```

Verificar que SSH escucha en el puerto 22:

```
sudo ss -tlnp | grep 22
```

Permitir SSH en el cortafuegos UFW:

```
sudo ufw allow ssh
sudo ufw enable
sudo ufw status verbose
```

4.4 Configuración del servidor SSH

El archivo de configuración principal del servidor SSH es `/etc/ssh/sshd_config`.

```
Port 22 # puerto de escucha (cambiar por seguridad)
PermitRootLogin no # IMPORTANTE: deshabilitar login de root
MaxAuthTries 3 # máximo de intentos de autenticación
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
PasswordAuthentication yes # cambiar a "no" cuando se usen claves
PermitEmptyPasswords no
X11Forwarding no # deshabilitar si no se necesita
ClientAliveInterval 300 # enviar keepalive cada 5 minutos
AllowUsers usuario1 usuario2 # solo estos usuarios pueden conectar
```

Después de modificar la configuración, verificar la sintaxis y reiniciar:

```
sudo sshd -t # verificar sintaxis sin reiniciar
sudo systemctl restart ssh
```

4.5 Autenticación por clave pública/privada

La autenticación por clave pública/privada es mucho más segura que la autenticación por contraseña. Se basa en criptografía asimétrica.

Generación del par de claves en el cliente

```
ssh-keygen -t ed25519 -C 'mi_email@ejemplo.com'
ssh-keygen -t rsa -b 4096 -C 'mi_email@ejemplo.com' # mayor compatibilidad
```

Copiar la clave pública al servidor

```
ssh-copy-id usuario@IP_SERVIDOR
```

Permisos correctos en el servidor

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

4.6 Hardening y seguridad SSH

Asegurar correctamente el servicio SSH es fundamental. A continuación se presentan las mejores prácticas de seguridad:

1. Cambiar el puerto por defecto

```
Port 2222 # o cualquier puerto alto no utilizado
sudo ufw allow 2222/tcp
```

2. Deshabilitar el login de root

```
PermitRootLogin no
```

3. Usar solo autenticación por clave

```
PasswordAuthentication no
PubkeyAuthentication yes
```

4. Usar Fail2Ban para proteger contra ataques de fuerza bruta

```
sudo apt install fail2ban -y
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
[sshd]
enabled = true
maxretry = 3
bantime = 3600      # bloqueo de 1 hora
sudo systemctl enable fail2ban && sudo systemctl start fail2ban
```

4.7 Túnel SSH y reenvío de puertos

El reenvío de puertos (port forwarding) es una de las funcionalidades más poderosas de SSH. Permite redirigir tráfico de red a través de una conexión SSH cifrada.

Reenvío local (Local Port Forwarding)

```
ssh -L puerto_local:host_destino:puerto_destino usuario@servidor
# Ejemplo: acceder a MySQL remoto como si fuera local
ssh -L 3307:localhost:3306 admin@192.168.1.100
```

Reenvío remoto (Remote Port Forwarding)

```
ssh -R puerto_remoto:localhost:puerto_local usuario@servidor
```

Proxy SOCKS dinámico

```
ssh -D 1080 usuario@servidor
```

4.8 SCP y SFTP: transferencia segura de archivos

SCP (Secure Copy Protocol)

```
scp archivo.txt usuario@servidor:/ruta/destino/
scp usuario@servidor:/ruta/archivo.txt ./
scp -r directorio/ usuario@servidor:/ruta/destino/
```

SFTP (SSH File Transfer Protocol)

SFTP es un protocolo de transferencia de archivos que funciona sobre SSH. A diferencia de SCP, ofrece una sesión interactiva similar a FTP pero completamente cifrada.

```
sftp usuario@servidor
sftp> get archivo.txt      # descargar archivo
sftp> put archivo.txt      # subir archivo
sftp> exit
```

5. SERVICIO DNS (DOMAIN NAME SYSTEM)

5.1 ¿Qué es DNS?

DNS (Domain Name System) es el sistema de nombres de dominio de Internet. Su función principal es traducir nombres de dominio legibles por humanos (como `www.ejemplo.com`) en direcciones IP numéricas (como `93.184.216.34`) que los ordenadores utilizan para comunicarse entre sí. Sin DNS, los usuarios tendrían que recordar las direcciones IP de cada sitio web o servicio.

El sistema DNS es jerárquico y distribuido. La jerarquía se organiza en niveles:

- Raíz (root): nivel más alto, representado por un punto (.) al final del nombre de dominio.
- TLD (Top-Level Domain): dominios de primer nivel como `.com`, `.org`, `.es`, `.net`.
- Dominio de segundo nivel: el nombre registrado, como `ejemplo.com`.
- Subdominio: niveles adicionales, como `www.ejemplo.com` o `mail.ejemplo.com`.

El proceso de resolución DNS involucra varios actores:

- Resolver recursivo: recibe la consulta del cliente y se encarga de obtener la respuesta final preguntando a los demás servidores.
- Servidor raíz: conoce los servidores autoritativos de cada TLD.
- Servidor TLD: conoce los servidores autoritativos de los dominios de segundo nivel.
- Servidor autoritativo: tiene la respuesta definitiva para los registros de un dominio concreto.

5.2 Tipos de registros DNS

Tipo	Descripción	Ejemplo
A	Asocia un nombre de dominio a una dirección IPv4	<code>www</code> → <code>192.168.1.10</code>
AAAA	Asocia un nombre de dominio a una dirección IPv6	<code>www</code> → <code>2001:db8::1</code>
CNAME	Alias que apunta a otro nombre de dominio	<code>ftp</code> → <code>www.ejemplo.com</code>
MX	Servidor de correo para el dominio	<code>mail.ejemplo.com</code> (prioridad 10)
NS	Servidores de nombres autoritativos del dominio	<code>ns1.ejemplo.com</code>
PTR	Resolución inversa: IP → nombre (zonas <code>in-addr.arpa</code>)	<code>10.1.168.192.in-addr.arpa</code> → <code>www</code>
SOA	Información de autoridad de la zona DNS	Serial, refresh, retry, expire
TXT	Texto libre; se usa para SPF, DKIM y verificaciones	<code>"v=spf1 mx ~all"</code>
SRV	Localización de servicios (VoIP, XMPP, etc.)	<code>_sip._tcp</code> : prioridad, peso, puerto, destino

5.3 Instalación de BIND9 en Ubuntu Server

BIND9 (Berkeley Internet Name Domain) es el servidor DNS más utilizado en el mundo. En Ubuntu Server se instala desde los repositorios oficiales:

```
sudo apt update
sudo apt install bind9 bind9utils bind9-doc -y
```

Verificamos que el servicio está activo y lo configuramos para que arranque con el sistema:

```
sudo systemctl status bind9
sudo systemctl enable bind9
sudo systemctl start bind9
```

Comprobamos que BIND9 escucha en el puerto 53 (UDP y TCP):

```
sudo ss -ulnp | grep 53    # UDP
sudo ss -tlnp | grep 53    # TCP
```

Permitimos el servicio DNS en el cortafuegos:

```
sudo ufw allow 53/udp
sudo ufw allow 53/tcp
```

Los archivos de configuración principales de BIND9 se encuentran en /etc/bind/:

- /etc/bind/named.conf: archivo de configuración principal (incluye los demás).
- /etc/bind/named.conf.options: opciones globales del servidor (recursión, forwarders, etc.).
- /etc/bind/named.conf.local: declaración de zonas DNS propias.
- /etc/bind/db.local: plantilla de zona de ejemplo.

5.4 Configuración de una zona DNS

A continuación se muestra cómo crear una zona DNS para el dominio ejemplo.local en un entorno de laboratorio.

Paso 1: Declarar la zona en named.conf.local

```
sudo nano /etc/bind/named.conf.local
zone "ejemplo.local" {
    type master;
    file "/etc/bind/db.ejemplo.local";
};
# Zona inversa (resolución inversa de la red 192.168.1.0/24)
zone "1.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/db.192.168.1";
};
```

Paso 2: Crear el archivo de zona directa

```
sudo cp /etc/bind/db.local /etc/bind/db.ejemplo.local
sudo nano /etc/bind/db.ejemplo.local
```

Contenido del archivo de zona directa:

```
; Zona directa para ejemplo.local
$TTL      604800
@ IN SOA ns1.ejemplo.local. admin.ejemplo.local. (
    2024010101 ; Serial (AAAMMDDnn)
    604800     ; Refresh (7 días)
    86400      ; Retry (1 día)
    2419200    ; Expire (28 días)
```



```

        604800 )      ; Negative Cache TTL
;
@   IN  NS    ns1.ejemplo.local.
@   IN  MX    10 mail.ejemplo.local.
ns1 IN  A     192.168.1.1
www  IN  A     192.168.1.10
mail IN  A     192.168.1.20
ftp  IN  CNAME www.ejemplo.local.

```

Paso 3: Crear el archivo de zona inversa

```

sudo cp /etc/bind/db.127 /etc/bind/db.192.168.1
sudo nano /etc/bind/db.192.168.1
; Zona inversa para 192.168.1.0/24
$TTL      604800
@   IN  SOA  ns1.ejemplo.local. admin.ejemplo.local. (
                        2024010101 ; Serial
                        604800 604800 86400 2419200 604800 )
@   IN  NS   ns1.ejemplo.local.
1   IN  PTR  ns1.ejemplo.local.
10  IN  PTR  www.ejemplo.local.
20  IN  PTR  mail.ejemplo.local.

```

Paso 4: Verificar la configuración y reiniciar

```

sudo named-checkconf          # verificar sintaxis global
sudo named-checkzone ejemplo.local /etc/bind/db.ejemplo.local
sudo systemctl restart bind9

```

5.5 DNS autoritativo y recursivo

BIND9 puede actuar como servidor autoritativo (contiene los datos de una zona), como servidor recursivo (resuelve consultas en nombre de los clientes consultando a otros servidores) o como ambos. Para un servidor de producción se recomienda separar estas funciones.

Configuración de forwarders en `named.conf.options` (para reenviar consultas no resueltas localmente):

```

options {
    directory "/var/cache/bind";
    recursion yes;
    allow-recursion { 192.168.1.0/24; localhost; }; # solo red local
    forwarders {
        8.8.8.8;    # Google DNS
        1.1.1.1;    # Cloudflare DNS
    };
    dnssec-validation auto;
    listen-on { any; };
};

```

5.6 Pruebas y verificación del servicio DNS

Herramientas esenciales para comprobar el funcionamiento del servidor DNS:

```

# Consulta directa
dig @192.168.1.1 www.ejemplo.local
nslookup www.ejemplo.local 192.168.1.1
# Consulta inversa (PTR)

```

```
dig @192.168.1.1 -x 192.168.1.10
# Ver los registros NS de una zona
dig @192.168.1.1 ejemplo.local NS
# Ver los registros MX de una zona
dig @192.168.1.1 ejemplo.local MX
# Comprobar el estado del servicio
sudo rndc status
sudo tail -f /var/log/syslog | grep named
```

6. SERVICIO HTTP (HYPERTEXT TRANSFER PROTOCOL)

6.1 ¿Qué es HTTP?

HTTP (HyperText Transfer Protocol) es el protocolo de comunicación base de la World Wide Web. Define cómo se deben formatear y transmitir los mensajes entre clientes (navegadores) y servidores web. Se trata de un protocolo sin estado (stateless), lo que significa que cada solicitud es independiente y el servidor no recuerda las solicitudes anteriores del mismo cliente.

Las versiones más relevantes del protocolo son:

- HTTP/1.0 (RFC 1945, 1996): versión básica con conexiones no persistentes. Cada recurso requiere una nueva conexión TCP.
- HTTP/1.1 (RFC 2616, 1997 – RFC 7230, 2014): introduce conexiones persistentes (keep-alive), pipelining y el campo Host en la cabecera, que permite los VirtualHosts.
- HTTP/2 (RFC 7540, 2015): multiplexación de solicitudes sobre una única conexión TCP, compresión de cabeceras (HPACK) y Server Push. Mejora significativamente el rendimiento.
- HTTP/3 (RFC 9114, 2022): utiliza QUIC (sobre UDP) en lugar de TCP, reduciendo la latencia y mejorando la recuperación ante pérdida de paquetes.

HTTP opera sobre el puerto 80 por defecto. Su versión segura, HTTPS, usa TLS/SSL y opera sobre el puerto 443.

6.2 HTTP vs HTTPS

Característica	HTTP	HTTPS
Puerto por defecto	80	443
Cifrado	No (texto plano)	Sí (TLS/SSL)
Certificado requerido	No	Sí (CA o autofirmado)
Integridad de datos	No garantizada	Garantizada mediante MAC
SEO (Google)	Penalizado	Favorecido
Indicador navegador	Advertencia "No seguro"	Candado verde / seguro
Velocidad	Ligeramente más rápido	Ligera sobrecarga por TLS (mínima en HTTP/2)
Recomendado	Solo entornos internos/dev	Siempre en producción

Para forzar la redirección de HTTP a HTTPS en Apache2:

```
<VirtualHost *:80>
    ServerName www.miweb.com
    Redirect permanent / https://www.miweb.com/
</VirtualHost>
```

6.3 Cabeceras HTTP relevantes

Las cabeceras HTTP permiten transmitir metadatos adicionales en las solicitudes y respuestas. Las más relevantes para la seguridad y el rendimiento son:

Cabecera	Tipo	Descripción
Content-Type	Respuesta	Tipo MIME del contenido (text/html, application/json...)
Content-Length	Respuesta	Longitud en bytes del cuerpo de la respuesta
Authorization	Solicitud	Credenciales de autenticación del cliente
Cache-Control	Ambos	Directivas para el control de caché
Strict-Transport-Security	Respuesta	Fuerza conexiones HTTPS (HSTS)
X-Frame-Options	Respuesta	Previene ataques de clickjacking
X-Content-Type-Options	Respuesta	Previene la detección errónea de tipo MIME
Content-Security-Policy	Respuesta	Define fuentes de contenido permitidas (CSP)
Set-Cookie	Respuesta	Establece una cookie; puede incluir Secure y HttpOnly
Location	Respuesta	URL de destino en redirecciones (3xx)

Configuración de cabeceras de seguridad en Apache2 (módulo mod_headers):

```
# Añadir al VirtualHost o a apache2.conf
Header always set Strict-Transport-Security "max-age=31536000;
includeSubDomains"
Header always set X-Frame-Options "SAMEORIGIN"
Header always set X-Content-Type-Options "nosniff"
Header always set Content-Security-Policy "default-src 'self'"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
```

6.4 Códigos de estado HTTP

Los códigos de estado HTTP indican el resultado de la solicitud del cliente. Se agrupan en cinco categorías:

Código	Categoría	Significado
200 OK	2xx Éxito	La solicitud se procesó correctamente
201 Created	2xx Éxito	Recurso creado exitosamente (POST/PUT)
204 No Content	2xx Éxito	Éxito sin cuerpo de respuesta
301 Moved Permanently	3xx Redirección	Redirección permanente a otra URL
302 Found	3xx Redirección	Redirección temporal
304 Not Modified	3xx Redirección	El recurso en caché sigue siendo válido
400 Bad Request	4xx Error cliente	Solicitud malformada o inválida
401 Unauthorized	4xx Error cliente	Se requiere autenticación
403 Forbidden	4xx Error cliente	Acceso denegado, sin permisos
404 Not Found	4xx Error cliente	Recurso no encontrado

500 Internal Server Error	5xx Error servidor	Error genérico del servidor
502 Bad Gateway	5xx Error servidor	Error en el servidor upstream (proxy)
503 Service Unavailable	5xx Error servidor	Servidor sobrecargado o en mantenimiento

6.5 Configuración de seguridad HTTP en Apache

Además de las cabeceras de seguridad, se recomienda aplicar las siguientes configuraciones en Apache2:

Ocultar la versión del servidor

```
# En /etc/apache2/conf-available/security.conf
ServerTokens Prod          # oculta la versión de Apache
ServerSignature Off        # elimina la firma en páginas de error
```

Deshabilitar el listado de directorios

```
<Directory /var/www/html>
    Options -Indexes
    AllowOverride All
</Directory>
```

Limitar los métodos HTTP permitidos

```
<LimitExcept GET POST HEAD>
    deny from all
</LimitExcept>
```

Configurar límites de solicitudes

```
LimitRequestBody 10485760 # máximo 10 MB por solicitud
LimitRequestFields 50     # máximo 50 cabeceras
Timeout 60                # tiempo máximo de espera
```

Después de los cambios, verificar la configuración y recargar Apache:

```
sudo apachectl configtest # verificar sintaxis
sudo systemctl reload apache2
```

7. SERVICIO DHCP (DYNAMIC HOST CONFIGURATION PROTOCOL)

7.1 ¿Qué es DHCP?

DHCP (Dynamic Host Configuration Protocol) es un protocolo de red definido en el RFC 2131 que permite a los dispositivos de una red obtener automáticamente su configuración IP sin necesidad de configuración manual. Cuando un dispositivo se conecta a la red, el servidor DHCP le asigna dinámicamente:

- Dirección IP.
- Máscara de subred.
- Puerta de enlace predeterminada (gateway).
- Servidores DNS.
- Otros parámetros opcionales (servidor NTP, nombre de dominio, etc.).

El proceso de asignación DHCP se denomina DORA y consta de cuatro pasos:

Paso	Mensaje	Descripción
1	DHCPDISCOVER	El cliente envía un mensaje de difusión (broadcast) para localizar servidores DHCP.
2	DHCPOFFER	El servidor responde con una oferta de configuración IP.
3	DHCPREQUEST	El cliente acepta la oferta y solicita formalmente la dirección IP.
4	DHCPACK	El servidor confirma la asignación y el cliente configura su interfaz de red.

Cada asignación tiene un tiempo de concesión (lease time) configurable. Antes de que expire, el cliente intenta renovarla. Si el servidor no está disponible, el cliente puede mantener la dirección durante el período de gracia. DHCP opera sobre UDP: el cliente usa el puerto 68 y el servidor el puerto 67.

7.2 Instalación de isc-dhcp-server en Ubuntu Server

El servidor DHCP más utilizado en Ubuntu es isc-dhcp-server, mantenido por la Internet Systems Consortium (ISC):

```
sudo apt update
sudo apt install isc-dhcp-server -y
```

Antes de iniciar el servicio, hay que indicar en qué interfaz de red escuchará el servidor DHCP. Editamos el archivo de configuración de la interfaz:

```
sudo nano /etc/default/isc-dhcp-server
# Indicar la interfaz de red (ejemplo: enp0s3 o eth0)
INTERFACESv4="enp0s3"
INTERFACESv6=""
```

Para conocer el nombre de la interfaz de red del servidor:

```
ip link show
ip addr
```

7.3 Configuración de rangos y opciones

El archivo principal de configuración del servidor DHCP es `/etc/dhcp/dhcpd.conf`. A continuación se muestra una configuración completa para la red 192.168.1.0/24:

```
sudo nano /etc/dhcp/dhcpd.conf
# Parámetros globales
default-lease-time 86400;          # tiempo de concesión por defecto: 1 día
max-lease-time 172800;             # tiempo de concesión máximo: 2 días
authoritative;                     # este servidor es el DHCP autorizado de la red
log-facility local7;               # registro en syslog

# Definición de la subred 192.168.1.0/24
subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200; # rango de IPs a asignar
    option routers 192.168.1.1;         # puerta de enlace
    option subnet-mask 255.255.255.0;   # máscara de subred
    option domain-name-servers 192.168.1.1, 8.8.8.8; # servidores DNS
    option domain-name "ejemplo.local"; # nombre de dominio
    option broadcast-address 192.168.1.255;
    option ntp-servers 192.168.1.1;     # servidor NTP (opcional)
}
```

Iniciamos y habilitamos el servicio:

```
sudo systemctl enable isc-dhcp-server
sudo systemctl start isc-dhcp-server
sudo systemctl status isc-dhcp-server
```

7.4 Reservas de IP (direcciones estáticas por MAC)

En muchas ocasiones es necesario asignar siempre la misma dirección IP a un dispositivo concreto (impresoras, servidores, cámaras IP, etc.). Esto se consigue mediante reservas basadas en la dirección MAC del dispositivo.

Para añadir una reserva, se agrega un bloque `host` dentro del archivo `dhcpd.conf`:

```
# Reservas de IP fija por dirección MAC
host servidor-web {
    hardware ethernet aa:bb:cc:dd:ee:01; # MAC del dispositivo
    fixed-address 192.168.1.10;          # IP reservada
    option host-name "servidor-web";     # nombre de host opcional
}

host impresora-plantal {
    hardware ethernet aa:bb:cc:dd:ee:02;
    fixed-address 192.168.1.50;
    option host-name "impresora-plantal";
}
```

Después de agregar reservas, verificar la sintaxis y reiniciar el servicio:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf # verificar sintaxis
sudo systemctl restart isc-dhcp-server
```

7.5 Verificación y registros

Para comprobar el correcto funcionamiento del servidor DHCP y consultar las concesiones activas:

```
# Ver el estado del servicio
```

```
sudo systemctl status isc-dhcp-server
# Ver los registros (logs) en tiempo real
sudo journalctl -u isc-dhcp-server -f
sudo tail -f /var/log/syslog | grep dhcp
# Ver las concesiones activas
cat /var/lib/dhcp/dhcpd.leases
# Herramienta de diagnóstico desde el cliente
sudo dhclient -v enp0s3          # solicitar IP al servidor DHCP
ip addr show enp0s3              # ver la IP asignada
```

Abrir los puertos necesarios en el cortafuegos (normalmente el servidor DHCP está en la misma red y no requiere reglas especiales de UFW, pero si hay múltiples interfaces:)

```
sudo ufw allow 67/udp      # puerto servidor DHCP
sudo ufw allow 68/udp      # puerto cliente DHCP
```


8. COMPARATIVA DE SEGURIDAD ENTRE SERVICIOS

La siguiente tabla compara los aspectos de seguridad más relevantes de los seis servicios estudiados:

Aspecto	FTP	Apache2	SSH	DNS	HTTP	DHCP
Cifrado por defecto	No	No/Sí (HTTPS)	Sí	No (DNSSEC opt.)	No/Sí (HTTPS)	No
Puerto por defecto	21/20	80/443	22	53	80/443	67/68
Autenticación	Usuario/contraseña	Múltiple	Contraseña/clave	TSIG (opcional)	Múltiple	MAC (opcional)
Versión segura	FTPS/SFTP	HTTPS + TLS	SSH-2	DNSSEC	HTTPS + TLS	DHCPv6 + IPsec
Fail2Ban recomendado	Sí	Sí	Sí	Parcialmente	Sí	No aplicable

9. BUENAS PRÁCTICAS Y RECOMENDACIONES

9.1 Buenas prácticas generales

- Mantener siempre el sistema operativo y los paquetes actualizados: `sudo apt update` && `sudo apt upgrade -y`
- Revisar regularmente los registros del sistema: `/var/log/syslog`, `/var/log/auth.log`, `/var/log/apache2/`, `/var/log/vsftpd.log`
- Usar el principio de mínimo privilegio: cada usuario y servicio debe tener solo los permisos estrictamente necesarios.
- Configurar el cortafuegos UFW correctamente, permitiendo solo los puertos necesarios.
- Realizar copias de seguridad periódicas de la configuración y los datos.
- Usar contraseñas fuertes o, mejor aún, autenticación por clave pública para SSH.
- Deshabilitar los servicios que no se usen.

9.2 Buenas prácticas específicas por servicio

FTP

- Usar siempre FTPS o sustituir FTP por SFTP.
- Deshabilitar el acceso anónimo.
- Encerrar a los usuarios en su directorio (`chroot`).
- Restringir el rango de puertos pasivos y abrir solo esos en el cortafuegos.

Apache2 / Web

- Forzar redirección de HTTP a HTTPS.
- Mantener los módulos innecesarios deshabilitados.
- Configurar cabeceras de seguridad: HSTS, X-Frame-Options, X-Content-Type-Options, CSP.
- Mantener PHP, WordPress y demás aplicaciones actualizadas.
- Usar `mod_security` como WAF (Web Application Firewall).

SSH

- Cambiar el puerto por defecto 22 por uno distinto.
- Deshabilitar el acceso de root por SSH.
- Usar exclusivamente autenticación por clave pública en producción.
- Instalar y configurar Fail2Ban.
- Usar solo SSH-2 y algoritmos modernos.

DNS

- Habilitar DNSSEC para proteger las respuestas DNS frente a ataques de envenenamiento de caché.
- Usar TSIG (Transaction Signature) para autenticar las transferencias de zona entre servidores maestro y esclavo.
- Separar el servidor DNS autoritativo del recursivo cuando sea posible.
- Limitar la recursión solo a los clientes de la red interna (`allow-recursion`).
- Mantener actualizado BIND9 para mitigar vulnerabilidades conocidas.

HTTP

- Forzar siempre HTTPS en producción y usar HSTS.
- Configurar las cabeceras de seguridad: CSP, X-Frame-Options, X-Content-Type-Options.
- Ocultar la versión del servidor (ServerTokens Prod).
- Deshabilitar el listado de directorios (Options -Indexes).
- Aplicar rate limiting para prevenir ataques de denegación de servicio (DoS).

DHCP

- Habilitar el registro de concesiones para auditoría y resolución de problemas.
- Usar reservas por MAC para dispositivos críticos de la red.
- Configurar tiempos de concesión adecuados según el entorno (cortos en redes públicas, largos en redes corporativas).
- Implementar DHCP snooping en los switches gestionados para prevenir servidores DHCP no autorizados.
- Considerar ISC Kea como alternativa más moderna a isc-dhcp-server.

10. REFERENCIAS Y BIBLIOGRAFÍA

A continuación se listan las referencias técnicas y normativas utilizadas para la elaboración de este documento:

- RFC 959 (1985) — Postel, J. y Reynolds, J.: "File Transfer Protocol". IETF.
- RFC 4217 (2007) — Ford-Hutchinson, P.: "Securing FTP with TLS". IETF.
- RFC 4251 (2006) — Ylönen, T. y Lonvick, C.: "The Secure Shell (SSH) Protocol Architecture". IETF.
- RFC 4252 — "The Secure Shell (SSH) Authentication Protocol". IETF.
- RFC 4253 — "The Secure Shell (SSH) Transport Layer Protocol". IETF.
- RFC 1035 (1987) — Mockapetris, P.: "Domain Names - Implementation and Specification". IETF.
- RFC 2131 (1997) — Droms, R.: "Dynamic Host Configuration Protocol". IETF.
- RFC 7230 (2014) — Fielding, R. y Reschke, J.: "Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing". IETF.
- RFC 7540 (2015) — Belshe, M. et al.: "Hypertext Transfer Protocol Version 2 (HTTP/2)". IETF.
- RFC 9114 (2022) — Bishop, M.: "HTTP/3". IETF.
- Documentación oficial de Ubuntu Server: <https://ubuntu.com/server/docs>
- Documentación oficial de OpenSSH: <https://www.openssh.com/manual.html>
- Documentación oficial de Apache HTTP Server: <https://httpd.apache.org/docs/>
- Documentación oficial de BIND9 (ISC): <https://bind9.readthedocs.io/>
- Documentación oficial de ISC DHCP: <https://www.isc.org/dhcp/>
- Página del proyecto vsftpd: <https://security.appspot.com/vsftpd.html>
- CIS Benchmarks para Ubuntu Linux: <https://www.cisecurity.org/>
- NIST SP 800-115: Technical Guide to Information Security Testing and Assessment.
- Documentación de Fail2Ban: <https://www.fail2ban.org/wiki/>

Nota: Toda la información de configuración presentada en este documento corresponde a Ubuntu Server 22.04 LTS (Jammy Jellyfish) con los paquetes disponibles en los repositorios oficiales de Ubuntu a fecha de 2024.